macro

Seven Seas A-41

Security Audit

May 5, 2025
Version 1.0.0

Presented by 0xMacro

Table of Contents

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled ‘Source Code’. The security audit was performed by the Macro security team over two days starting on April 25th, 2025.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro’s review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	1	-	-	1
Low	2	-	-	2

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Source Code

The following source code was reviewed during the audit:

- **Mellow's dvStETH vault decoder (from PR 251)**

  Commit Hash: `478f3a4a1457fd2dd296fc48d43ad5689998cc4a`

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/DvStETHDecoderAndSanitizer.sol	`38baba1ff4fbc624b2287956735fab8c1cef679b588c0ac6f0b1b72f92c6db41`

- **Generic rate provider with decimal scaling (from PR 254)**

  Commit Hash: `63b8a392581db533ad64a5371bef3a17d6a746d7`

Source Code	SHA256
src/helper/GenericRateProviderWithDecimalScaling.sol	`b5fa65ad59eb27e216e88b294be5909db2f9775b09a77de4bd295190864add26`

- **Teller Decoder fix (from PR 261)**

  Commit Hash: `75a57946ea4d4f8416c34bc18ab13c42f07ee3de`

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/TellerDecoderAndSanitizer.sol	`a60d2e60d92d27c3b9eb28438862f2b7f21e0196c6560f7fea2eed13a50f2b00`

- **Permissioned transfers for teller (from PR 256)**

  Commit Hash: `b719f64af56f49781ca74604ec50ea7927818aa2`

Source Code	SHA256
src/base/Roles/TellerWithMultiAssetSupport.sol	`b4f6b373696ce73a739901bc5b1d118b5d83ccd9282b840a08ef25b9c9a1647b`

- **Beraborrow decoder (from PR 258)**

  Commit Hash: `e70b91e6173ac13d15038c05473bc5b0a8d47a1c`

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/BeraborrowDecoderAndSanitizer.sol	`f57f1547b5472b8fdc12f4358829ffd3316a0804c6aa58e20cab5ed216e6ad36`

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:
    - How bad things can get (for a vulnerability)
    - The significance of an improvement (for a code quality issue)
    - The amount of gas saved (for a gas optimization)
2. The high/medium/low **likelihood** of the issue:
    - How likely is the issue to occur (for a vulnerability)
3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client **must** fix the issue, no matter what, because not fixing would mean **significant funds/assets WILL be lost.**
(H-x) High	We recommend the client **must** address the issue, no matter what, because not fixing would be very bad, *or* some funds/assets will be lost, *or* the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to **seriously consider** fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albiet not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

~~M-1~~ Incorrect sanitization of destination chain

TOPIC	STATUS	IMPACT	LIKELIHOOD
Validation	Fixed Fixed	Medium	Medium

The assembly blocks in both the merkle tree generation and the decoder's `depositAndBridge` function aim to convert a `uint32` value from a `bytes` array to an `address` type. However, due to the final right shift operation (`shr(96, word)`), the conversion always returns `0x0` for the chain ID. The test appears to pass only because both functions return `0x0` for the address, meaning the sanitization is ineffective and allows any chain ID to be passed.

Remediation to Consider

Remove the right shift operation from the assembly blocks to preserve the actual converted value.

~~L-1~~ Mellow decoder doesn’t sanitize the deposit token

TOPIC	STATUS	IMPACT	LIKELIHOOD
Validation	Fixed	Medium	Low

In Mellow's deposit function, the parameter `amounts` determines which underlying vault tokens are being deposited based on their index in the array. While currently only `WETH` and `wstETH` tokens are supported, protocol admins can add additional tokens through the Vault's `addToken` function. To maintain control over which tokens can be deposited, the boring vault needs to sanitize the underlying tokens based on their index in the `amounts` array. The likelihood of this issue is low since token deposits are additionally restricted by token approvals.

Remediation to Consider

Retrieve the vault's underlying tokens through the `underlyingTokens()` function and sanitize them according to their corresponding positions in the `amounts` array.

~~L-2~~ `preDeposits` can be passed in `adjustDenVault`

TOPIC	STATUS	IMPACT	LIKELIHOOD
Validation	Fixed	Medium	Low

In the Beraborrow decoder's `openDenVault` function, passing `preDeposit` is explicitly disallowed by this check:

```
if (params._preDeposit.length > 0) revert BeraborrowDecoderAndSanitizer__Predep
```

However, the `adjustDenVault` function lacks this check, which allows values to be passed to the `_preDeposit` parameter. This could enable deposits of assets not explicitly allowed by the boring vault.

Remediation to Consider

Add the above check to the `adjustDenVault` function.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.